FAST – NUCES

Object Oriented Programming Lab Manual

Course Teacher	Ms. Hina Iqbal
Lab Instructor	Mr. Ahmad Jawad Mustasim
Lab TA	Ms. Mania Shakeel
Section	BSE-2B
Semester	Spring 2026

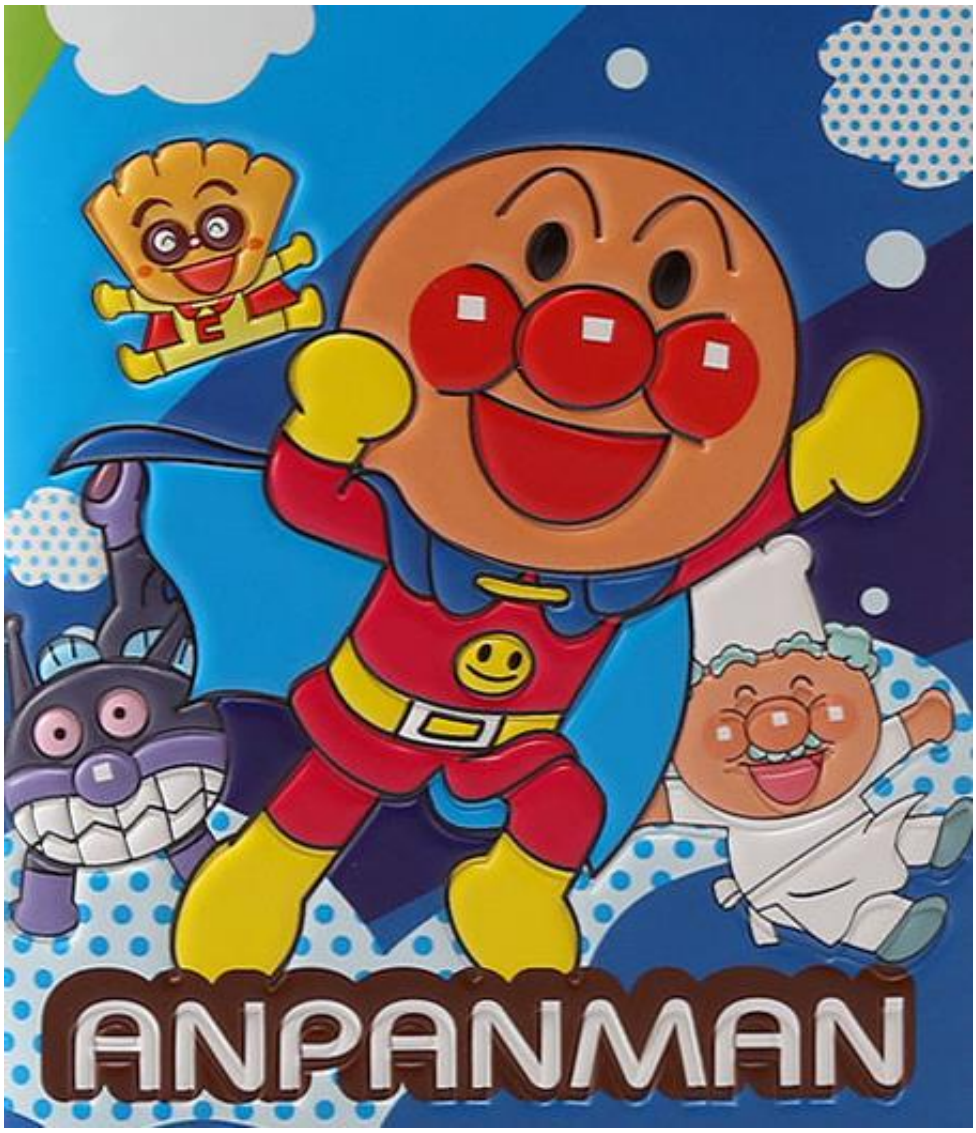
Department of Computer Science

FAST-NU, Lahore, Pakistan

Lab 8 – Deep Copy, Static Members and Operator Overloading

Concepts Covered

- deep copy
- move constructor
- static data members
- static member functions
- cascading setters
- operator overloading (unary ++ and binary +)
- dynamic arrays of objects



Read carefully and attempt following task

In the **Anpanman universe**, Anpanman is building intelligent **Medical Helper Bots** that assist injured citizens across the town.

Each bot has:

- a **name**
- a **healing capacity**

Your task is to design a **C++ program using Object Oriented Programming concepts** to manage these bots.

Students must implement a class named:

AnpanMedBot

Class Requirements

Private Data Members

1. char* name

- Stores the name of the bot
- Must be **dynamically allocated**
- string class is **NOT allowed**

2. int healingCapacity

Represents the healing power of the bot.

Valid range:

0 – 1000

Example meaning:

Range	Bot Strength
0–100	Weak helper

Range	Bot Strength
100–400	Normal helper
400–800	Advanced helper
800–1000	Elite medical bot

Static Members

Static Data Member

static int totalBots;

Purpose:

Tracks the **total number of bots created**.

Static Member Function

static void showTotalBots();

Displays the total number of bots created.

Example output:

Total Bots Created: 5

Constructors

1. Default Constructor

Initialize:

name → "UnknownBot"
healingCapacity → 0

Requirements:

- Allocate dynamic memory for name
- Copy characters **manually using loops**
- Increment totalBots

2. Overloaded Constructor

Parameters:

const char* name
int healingCapacity

Requirements:

- Dynamically allocate memory for name
- Copy characters manually/ Deep Copy
- Validate healing capacity
- Increment totalBots

3. Copy Constructor

Create a **deep copy** of the object.

Requirements:

- Allocate **new memory** for name
- Copy characters manually
- Ensure both objects **do not share the same memory**

Example concept:

AnpanMedBot bot2(bot1);

Both bots must have **separate memory for name**.

4. Move Constructor

Students must implement a **move constructor**.

Purpose:

Transfer ownership of dynamic memory instead of copying it.

Prototype:

AnpanMedBot(AnpanMedBot&& other);

Requirements:

- Transfer the pointer from other to the new object
- Set other.name to nullptr
- Copy healingCapacity
- Avoid unnecessary memory allocation

Concept example:

AnpanMedBot bot2 = std::move(bot1);

After move:

- bot2 owns the memory
- bot1.name becomes nullptr

Cascading Setter Functions

Students must implement **method chaining setters**.

Pointer Cascading Setter

AnpanMedBot* setName(const char* n);

Returns:

this

Example usage:

bot.setName("CareBot")->setHealingCapacity(300);

Reference Cascading Setter

AnpanMedBot& setHealingCapacity(int h);

Returns:

*this

Example usage:

bot.setHealingCapacity(200).setHealingCapacity(500);

Complex Chaining Practice

Students must test expressions like:

bot.setName("HeroBot")->setHealingCapacity(300).setHealingCapacity(600);

(*(*bot.setName("MegaCare")).setHealingCapacity(200).setHealingCapacity(500)).setHealingCapacity(800);

These help practice **pointer and reference chaining**.

Unary Operator Overloading (Member Function)

Students must overload **increment operator ++**.

Purpose:

Increase healing power of the bot.

Example usage:

++bot;

Effect:

healingCapacity += 50

Maximum allowed:

1000

Prototype:

void operator++();

Binary Operator Overloading (Member Function)

Students must overload **binary operator +**.

Purpose:

Combine healing power of two bots.

Example usage:

bot3 = bot1 + bot2;

Behavior:

bot3.healingCapacity =
bot1.healingCapacity + bot2.healingCapacity

Prototype:

AnpanMedBot operator+(const AnpanMedBot& other);

The operator should return a **new bot object**.

Getter Functions

Students must create:

getName()
getHealingCapacity()

These functions simply **return stored values**.

Display Function

void display();

Output format:

Bot Name: CareHelper
Healing Capacity: 550

Input Validation Rules

Students must ensure:

- Name cannot be empty
- Healing capacity must be **≥ 0**
- Healing capacity must not exceed **1000**
- Dynamic memory must be handled safely

Main Function Requirements

1. Ask User for Number of Bots

Example prompt:

Enter number of Medical Helper Bots:

Validation rule:

number must be greater than 0

2. Create Dynamic Array of Objects

Example:

```
AnpanMedBot* bots = new AnpanMedBot[size];
```

3. Input Data for Each Bot

For each bot:

Enter bot name:
Enter healing capacity:

Name must be entered using **getline**.

4. Store Values Using Cascading Setters

Example:

```
bots[i].setName(tempName)->setHealingCapacity(capacity);
```

5. Display All Bots

Use:

```
display();
```

6. Demonstrate Operator Overloading

Unary increment example:

++bots[0];

Binary addition example:

AnpanMedBot superBot;

superBot = bots[0] + bots[1];

Display the resulting bot.

7. Display Total Bots

Call static function:

AnpanMedBot::showTotalBots();

Restrictions

Students **must NOT use**:

- string
- strcpy
- strlen
- strcmp
- any built-in copy functions

----------Happy Coding ☺----------

----------Submission Instructions----------

- Submit your tasks on Google Classroom within given deadline
- Always submit .cpp files when asked for submission
- Do all tasks in single .cpp file